SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1711**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 2 Duration: 1 Hour Total Marks:20

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I **S. Bhuvanesh (192521004)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in

disciplinary action. ***Signature of the student:***

S. Bhuvanesh (1092521004)

CO1 Assessment: Constraint-Based Problem Solving

Question 4. Map Colouring of South Indian States (CSP with Heuristics)

Problem: Colour TN, KL, KA, AP, TG, GA and MH with {Red, Green, Blue} so that no two neighbouring states share a colour. Neighbouring pairs: TN–KL, TN–KA, TN–AP, KL–KA, KA–AP, KA–TG, KA–GA, KA–MH, AP–TG, GA–MH, MH–TG.

(i) CSP formulation and constraint graph

- **Variables:** X = {TN, KL, KA, AP, TG, GA, MH}
- **Domains:** D(X) = {Red, Green, Blue} for every variable X.
- **Constraints:** 11 binary "not-equal" constraints, one per neighbouring pair:

TN ≠ KL, TN ≠ KA, TN ≠ AP, KL ≠ KA, KA ≠ AP, KA ≠ TG, KA ≠ GA, KA ≠ MH, AP ≠ TG, GA ≠ MH, MH ≠ TG.

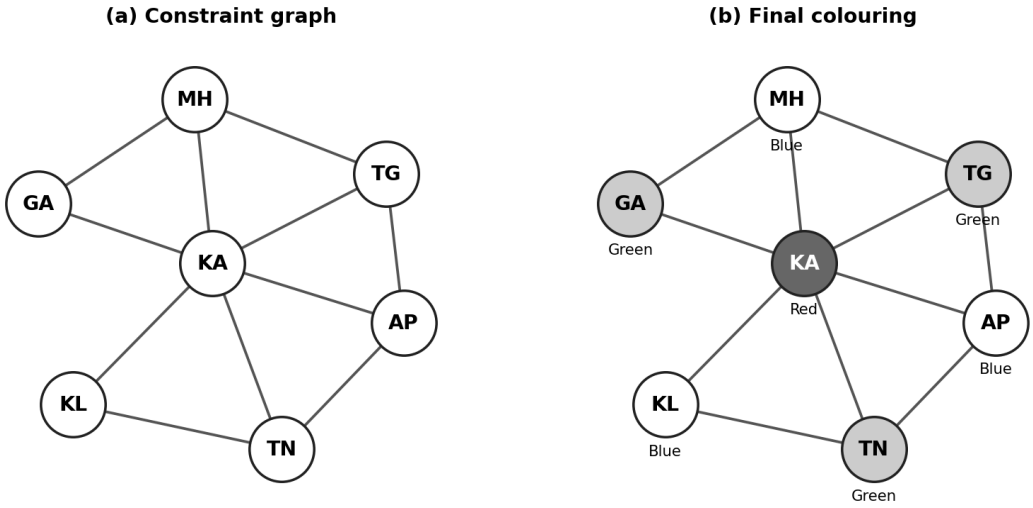


Figure 1. (a) Each state is a node and each neighbouring pair is an edge. (b) Dark grey = Red, light grey = Green, white = Blue.

(ii) Variable-ordering heuristics

Degree table (number of constraints on each variable):

State	Neighbours	Degree
KA	TN, KL, AP, TG, GA, MH	6
TN	KL, KA, AP	3
AP	TN, KA, TG	3
TG	KA, AP, MH	3
MH	KA, GA, TG	3
KL	TN, KA	2
GA	KA, MH	2

- **Degree heuristic (first variable):** KA has the highest degree (6), so it is assigned first. It constrains the most other variables.
- **MRV (remaining variables):** at each step pick the unassigned variable with the fewest legal values left. Ties are broken by the larger number of unassigned neighbours (degree), then by the order TN, KL, AP, TG, GA, MH. Values are tried in the order Red, Green, Blue.

(iii) Backtracking with forward checking

R = Red, G = Green, B = Blue. Bold letter = assigned value; cells in braces show the remaining domain after forward checking removes the assigned colour from the neighbours.

Step	Variable chosen and reason	TN	KL	KA	AP	TG	GA	MH
0	Start (no assignment)	{R,G,B}	{R,G,B}	{R,G,B}	{R,G,B}	{R,G,B}	{R,G,B}	{R,G,B}
1	KA: highest degree (6) → KA = Red	{G,B}	{G,B}	**R**	{G,B}	{G,B}	{G,B}	{G,B}
2	MRV tie (all size 2); tie-break: most unassigned neighbours (TN=2) → TN = Green	**G**	{B}	**R**	{B}	{G,B}	{G,B}	{G,B}
3	MRV: KL{B}, AP{B}; AP has 1 unassigned neighbour (TG) → AP = Blue	**G**	{B}	**R**	**B**	{G}	{G,B}	{G,B}
4	MRV: TG{G}, KL{B}; TG has unassigned neighbour MH → TG = Green	**G**	{B}	**R**	**B**	**G**	{G,B}	{B}
5	MRV: MH{B}, KL{B}; MH has unassigned neighbour GA → MH = Blue	**G**	{B}	**R**	**B**	**G**	{G}	**B**
6	MRV: KL{B}, GA{G} (tie) → KL = Blue (first in order)	**G**	**B**	**R**	**B**	**G**	{G}	**B**

Step	Variable chosen and reason	TN	KL	KA	AP	TG	GA	MH
7	Only GA{G} left → GA = Green	**G**	**B**	**R**	**B**	**G**	**G**	**B**

Result of the search: 7 assignments, 0 backtracks. No domain became empty at any step, so forward checking never detected a dead end.

(iv) Final colouring and effect of the heuristics

State	Colour	Check against neighbours
KA	Red	TN, KL, AP, TG, GA, MH are all Green/Blue ✓
TN	Green	KL Blue, KA Red, AP Blue ✓
KL	Blue	TN Green, KA Red ✓
AP	Blue	TN Green, KA Red, TG Green ✓
TG	Green	KA Red, AP Blue, MH Blue ✓
GA	Green	KA Red, MH Blue ✓
MH	Blue	KA Red, GA Green, TG Green ✓

All 11 constraints are satisfied, so this is a valid 3-colouring: **KA = Red; TN, TG, GA = Green; KL, AP, MH = Blue.**

How the heuristics reduced backtracking:

- **Degree heuristic:** KA touches 6 of the 7 states. Colouring it first means every other state loses one colour immediately, so the hardest decision is made while all choices are still open.
- **MRV:** after KA, MRV kept choosing states whose domain had shrunk to a single colour. These are forced moves, so no wrong guess was possible and the chain TN → AP → TG → MH → GA fixed itself.
- **Forward checking:** it removes the conflicting colour from neighbours right after each assignment, so a dead end shows up as an empty domain at once.
- **Contrast with a poor ordering:** if GA = Red, MH = Green and TG = Blue were chosen first (legal, since GA and TG are not neighbours), KA would be adjacent to all three colours and its domain would become empty. The search would have to backtrack. Choosing KA first avoids this.

Question 7. Ambulance Routing During Floods (A* Search with a Road-Closure Constraint)

Problem: Find the fastest route from accident site S to hospital G when road B–D is closed. Costs are travel times in minutes; heuristic h(n) is the straight-line time to G.

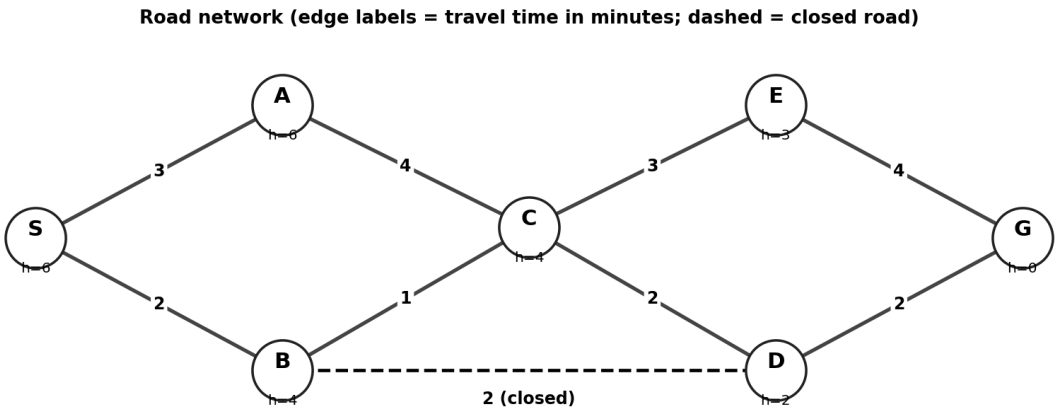


Figure 2. Road network with heuristic values. The closed road B–D is excluded from the search.

(ii) A* search with the road B–D closed

Evaluation function: f(n) = g(n) + h(n), where g(n) is the cost from S to n. The OPEN list is ordered by smallest f. Usable roads give neighbours: S{A,B}, A{S,C}, B{S,C}, C{A,B,D,E}, D{C,G}, E{C,G}, G{D,E}.

Step	Node expanded	Successors generated (g + h = f)	OPEN after step (g + h = f)	CLOSED
0	–	Start node S: 0 + 6 = 6	S(0+6=6)	{ }
1	S (f=6)	A: 3+6=9; B: 2+4=6	B(2+4=6), A(3+6=9)	{S}

Step	Node expanded	Successors generated (g + h = f)	OPEN after step (g + h = f)	CLOSED
2	B (f=6)	C: 3+4=7 (via B); D not reachable (B–D closed); S already closed	C(3+4=7), A(3+6=9)	{S, B}
3	C (f=7)	D: 5+2=7; E: 6+3=9; A via C: 7+6=13 (worse than 9, discarded); B closed	D(5+2=7), A(3+6=9), E(6+3=9)	{S, B, C}
4	D (f=7)	G: 7+0=7; C already closed	G(7+0=7), A(3+6=9), E(6+3=9)	{S, B, C, D}
5	G (f=7)	Goal selected from OPEN, so search stops	–	{S, B, C, D, G}

(ii) Route found and total cost

Following parent pointers back from G (G ← D ← C ← B ← S):

Route: S → B → C → D → G Cost = 2 + 1 + 2 + 2 = **7 minutes**.

Only four nodes (S, B, C, D) were expanded; A and E were never expanded. The other routes are costlier: S–B–C–E–G = 10 and S–A–C–D–G = 11.

(iii) A* if road B–D were open

Step	Node expanded	Successors generated (g + h = f)	OPEN after step (g + h = f)	CLOSED
0	–	Start node S: 0 + 6 = 6	S(0+6=6)	{ }
1	S (f=6)	A: 3+6=9; B: 2+4=6	B(2+4=6), A(3+6=9)	{S}
2	B (f=6)	C: 3+4=7; D: 4+2=6 (via B–D)	D(4+2=6), C(3+4=7), A(3+6=9)	{S, B}
3	D (f=6)	G: 6+0=6; C via D: 6+4=10 (worse than 7, discarded)	G(6+0=6), C(3+4=7), A(3+6=9)	{S, B, D}
4	G (f=6)	Goal selected from OPEN, so search stops	–	{S, B, D, G}

Route: S → B → D → G Cost = 2 + 2 + 2 = **6 minutes**. The flood closure therefore adds 1 minute (7 vs 6) and forces the detour through C.

(iv) Admissibility and consistency of h for the flooded network

Admissibility: $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cheapest cost from n to G using only open roads.

Node	h(n)	h*(n)	Cheapest path to G	h(n) ≤ h*(n)?
S	6	7	S–B–C–D–G (2+1+2+2)	6 ≤ 7 ✓
A	6	8	A–C–D–G (4+2+2)	6 ≤ 8 ✓
B	4	5	B–C–D–G (1+2+2); B–D is closed	4 ≤ 5 ✓
C	4	4	C–D–G (2+2)	4 ≤ 4 ✓
D	2	2	D–G (2)	2 ≤ 2 ✓
E	3	4	E–G (4)	3 ≤ 4 ✓
G	0	0	–	0 ≤ 0 ✓

Consistency: $h(n) \le c(n, n') + h(n')$ for every road, checked in both directions.

Road	c	$n \to n'$	$n' \to n$
S–A	3	$h(S)=6 \le 3+6=9$ ✓	$h(A)=6 \le 3+6=9$ ✓
S–B	2	$h(S)=6 \le 2+4=6$ ✓	$h(B)=4 \le 2+6=8$ ✓
A–C	4	$h(A)=6 \le 4+4=8$ ✓	$h(C)=4 \le 4+6=10$ ✓
B–C	1	$h(B)=4 \le 1+4=5$ ✓	$h(C)=4 \le 1+4=5$ ✓
C–D	2	$h(C)=4 \le 2+2=4$ ✓	$h(D)=2 \le 2+4=6$ ✓
C–E	3	$h(C)=4 \le 3+3=6$ ✓	$h(E)=3 \le 3+4=7$ ✓
D–G	2	$h(D)=2 \le 2+0=2$ ✓	$h(G)=0 \le 2+2=4$ ✓
E–G	4	$h(E)=3 \le 4+0=4$ ✓	$h(G)=0 \le 4+3=7$ ✓

Conclusion: the heuristic is both admissible and consistent for the flooded network. Closing B–D can only raise true costs ($h^*(B)$ goes from 4 to 5 and $h^*(S)$ from 6 to 7) and removes one road from the consistency check, so h stays valid. Because h is consistent, f never decreases along the route (6, 6, 7, 7, 7), closed nodes never need re-opening, and the first time G is selected the route is optimal. This matches the 7-minute result in part (ii).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	0	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided